Vision-based Air-to-Air Unmanned Aerial Vehicle Detection using YOLO based Architectures

Dapinder Kaur[1,2], Neeraj Battish[1], Arnav Bhavsar[3], Shashi Poddar[*,1,2]

[1]CSIR – Central Scientific Instruments Organisation, Sector 30C, Chandigarh
[2]Academy of Scientific & Innovative Research, CSIR – CSIO Campus, Ghaziabad
[3]IIT Mandi, Himachal Pradesh
*shashipoddar@csio.res.in

ABSTRACT: Unmanned aerial vehicles (UAVs) are getting widely popular in recent times due to their diverse applications and have given rise to security issues associated with them. Vision-based systems for detecting UAV use fixed or moving cameras on the ground or in the air. Ground-to-Air (G2A) UAV detection uses a machine learning approach and has been showing good accuracy in clear background and weather conditions, while Air-to-Air (A2A) UAV detection is still challenging because of the complex environmental scenarios. Therefore, the focus of this work is to study and analyse the vision-based approaches for detecting UAV in an A2A scenario using different deep learning methods. Among the deep learning approaches, the conventional techniques are multi-stage detectors which generally use a sliding window approach while the YOLO-based technique uses a single-stage detector. This work analyzes the performance of different YOLO architectures YOLOv3, YOLOv4, YOLOv5, YOLOv6 and YOLOv7 for the task of A2A UAV detection. The dataset of A2A UAV- DeTFly is used for testing the performance of different YOLO models with the same training parameters in terms of accuracy and precision parameters. The implemented models are also evaluated based on their size, prediction time, etc. to understand the performance of different YOLO models.

Keywords: UAV detection, Air-to-Air, deep learning, YOLO, DeTFly

1. Introduction

Unmanned Aerial Vehicles (UAVs) have progressed rapidly in the last decade due to their applications in agriculture, medicine, filmmaking, and many more industries. Its market is also expanding for government and military applications, where border security, crime prevention, disaster management, etc., are some of the typical military uses [1]. These UAVs, being agile, and dynamic make it a suitable candidate for launching malicious attacks from distance. Recent security threats at defense premises have led to an immediate demand for autonomous systems that can secure the building perimeters by detecting these flying objects. With the rise of UAVs in different domains, a constant increase in threat and potential sightings of UAVs have led to major disturbances on the public infrastructures across the world. For example, drone sighting at Gatwick Airport, UK in 2018, Frankfurt airport, Germany in 2019, Jammu Airbase, India is some of the recent disturbances. Similarly, attacks on individuals/ groups of individuals using drones have become very common. For example, in January 2015, a UAV crashed on the White House lawn, raising concerns about existing safety measures [2] an army chopper was damaged by an unlicensed drone in September 2017 [3], Mexican gangs have been smuggling pharmaceutical supplies across the border in the USA, and illegal smartphone trade across Hong Kong and China is very commonly reported in international news [4]. Several such drone incidents have also been reported in India wherein arms and drug smuggling happen across the border through unidentified drones. These kinds of events are not just detrimental to the progress of any country but also raise several security concerns which need to be addressed holistically. To ensure the security of strategic infrastructure, some locations have been tagged as a no-flying zone, forbidding drones from flying within a specified radius of sensitive areas such as airports, prisons, power plants, and other critical infrastructure. However, these no-fly zones need to have a mechanism by which they can always monitor the presence of any intruding drone. As a result, the development of anti-drone systems is gaining pace across the world, and the problem of real-time drone detection is becoming more relevant [5].

The problem of UAV detection can be classified into two categories: (i) Ground-to-Air (G2A) UAV detection; and (ii) Air-to-Air (A2A) UAV detection. In G2A, the camera is on the ground that may be fixed or moving slowly, focusing on the air to capture or detect UAVs. In this case, the background for the target UAV image is a clear or cloudy sky and will not have man-made structures in its background in most of the cases. On the other hand, A2A is more complex because both the detector and the object in this case are in the air and can have multiple view angles to capture the UAV. Moreover, the background of the target UAV image could be a highly complex scene such as urban environment, natural fields, etc. In this paper, the research focus is on A2A UAV Detection which has different backgrounds, view angles, motion blurring, varying lighting conditions, and many more [6].

Various researchers earlier considered the detection of UAVs in the G2A scenario using sensors like LIDAR, IMU, radio Systems, Radar, Cameras, etc.[7]. Garcia et al. [8] used a Faster-RCNN model with ResNet-101 backbone and tested the model on a SafeShore project dataset. This dataset was created using 3D-LIDAR with a passive acoustic sensor, radio detector, and video analyzation which covers a distance of 1500m to 1800m. The proposed model was compared with the RCNN and Fast RCNN models, and it was found that the performance of Faster-RCNN was highest, with 93.4% accuracy. Moreover, a camera and

IMU sensor-based tracking and localization of the drone swarm were done by pavliv et al. (2021) [9]. A multisensor fusion using a radio system, radar, and an ADS-B transponder with four antenna arrays was also proposed in [10]. RF-signal based data was used in a deep learning approach for detecting and classifying UAVs in [11]. Martins et al. [12] used basic image processing steps like blurring, edge detection, and contour finding for UAV detection while Demir et al. (2019) [13] used background subtraction with morphological operations for the same. Moreover, machine learning algorithms, including decision tree, SVM, KNN, Naïve Bayes, K-Means, Boosted Trees, HMM, GMM, etc., were explored for UAV detection using radar, acoustic, visual, and RF-signal-based sensing [14].

The main contribution of this paper is to develop a model for A2A UAV detection using YOLO-based algorithms and to compare their performances on an identical set of training parameters. Five different YOLO models, YOLOv3, YOLOv4, YOLOv5, YOLOv6, and YOLOv7, are trained for the A2A UAV detection and tested on DeTFly dataset [6]. Although YOLOv3, v4 and v5 models and their variations have been applied for the task of UAV detection in the past, they have not been tested against each other under similar conditions. The application of newer versions of YOLO architectures, that is YOLO v6 and v7 is also attempted here for the task of A2A UAV detection. To determine the best model for this specific scenario based on accuracy, speed, and resource needs, these YOLO models for A2A UAV detection are compared thoroughly which has not been carried out earlier to the best of the author's knowledge. As the YOLO variations have their own advantages and disadvantages, it is crucial to assess these models' performance and provide a detailed analysis on them for A2A UAV detection tasks. Researchers and practitioners can choose the most suitable model for their application requirements by contrasting several YOLO models to better understand their strengths and limitations and this analysis can be used for other detection architectures as well. The article is divided into four sections of which section 2 provides a brief overview of the literature in the field of UAV detection in both the G2A and A2A categories, section 3 analyses different YOLO architectures and its evolution along with the improvements carried out in each of them in the past, section 4 compares different YOLO model in terms of their head, neck and backbone structure, activation and loss function and finally section 5 provides a detailed experimental analysis on different YOLO models on the DetFly dataset.

2. Vision based UAV Detection using Deep Learning

This section of the paper overviews the existing methods developed for detecting UAVs using vision-based approaches with deep learning. As discussed in the previous section, the detection of UAVs can be classified into two sub-sections depending on the location of the camera with respect to the UAV, that is, Ground to Air (G2A) and Air to Air (A2A). The following sub-sections deliberate on the existing techniques for both G2A and A2A.

2.1 Ground-to-Air (G2A) UAV Detection

Several approaches have been developed in the past to detect the existence of UAV using a static or a moving camera on the ground. Jamil et al. (2022) [15] proposed a vision transformer-based framework called ViT to identify different aerial vehicles. It extracted handcrafted features like HoG, LBP, GLCM, CJLBP, etc., and used these features to train the classifier. The ViT framework used different DCNN models, including, AlexNet, MobileNet, ShuffleNet, Darklnet-53, etc., for feature extraction followed by a multi-layer perceptron network classification layer. In this work, they work on five different classes of air objects: birds, planes, helicopters, drones, and malicious drones. In total, they have 776 images in the dataset, of which 70% are used for training and the rest for testing. Dil et al. (2022) [16] proposed a MultiFeatureNet (MFNet) in which the architecture is modified by increasing or decreasing the layers' kernel size to improve the model's feature extraction and overall performance. This model was tested on a publicly available dataset of 5105 images, including drones and birds.

Javan et al. (2022) [17] proposed a novel technique based on improved YOLOv4 architecture to recognize drones and birds. In this proposed YOLOv4 architecture, three convolutional layers are added after the backbone for feature extraction and to deal with spatial redundancy. Also, two convolutional layers were added to the head to reduce the network computation's final depth and volume. They created the dataset from the publicly available images and used 70% for training. The results show that an accuracy of 83% was achieved with the proposed model, whereas existing YOLOv4 gives only 79% accuracy. Zhao et al. (2022) [18] proposed a UAV dataset named DUT Anti-UAV with around 10000 images and 20 videos. They have implemented different detection algorithms like Faster-RCNN, Cascade-RCNN, ATSS, YOLOX, and SSD with different backbones and found that the performance of Cascade-RCNN with ResNet50 is better than all other models in terms of mAP. Kabir et al. [19] implemented YOLOv4, YOLOv5, and Detection Transformer (DETR) models to detect UAVs. They have prepared their dataset of 2000 images from the existing datasets and performed pre-processing to remove artifacts. They used 70% for training, 20% for validation, and only 10% for testing the models. The detection results in AP show the effectiveness of the DETR model compared to the YOLOv4 and YOLOv5 models.

Various other approaches, such as an active vision-based relative localization framework were proposed to tackle the restricted FOV [20]. Measurements from active vision, visual inertial odometry, and UWB were fused to obtain relative positions, achieving 5 cm RMSE at different velocities. In order to deal with the cluttered background, shadows, and reflections while detecting the UAV based on images, Trapal et al. [21] proposed an improved approach that increases the resolution of images with a shadow removal algorithm. Another issue related to the environment for vision-based detection under sunlight was also considered, and an approach based on contrast and HSV-based conversion was proposed in [22]. YOLOv3-based detection algorithm was used to detect objects and to sense and avoid collision between two UAVs. Kalman filter, blind spot algorithm, and velocity obstacle algorithm were used in [23]. Other deep learning algorithms, including YOLOv2 [23],YOLOv3 [6], MobileNetV2 CNN [5], TIB-Net [24], ResNet50 with SVM [25], Improved

CNN [26] were developed for detection of UAV from ground .

2.2 Air-to-Air (A2A) UAV Detection

In the Air-to-Air UAV detection scenarios, quite a few works have been reported in the literature and are mostly based on deep learning architectures. Recently, Sangam et al. (2022) [27] proposed a deep learning model that uses CSP-Darknet53 and Video Swin Model for drone-to-drone detection. They elaborate on the challenges of this detection problem and discuss the main issues, including small object size, both target and source movement, and uncontrolled surroundings. In this model, they perform temporally consistent augmentation on the video frames. These inputs are given to the backbone-CSP Darknet-53, and then spatial features are extracted using spatiotemporal swin transform and attention model. Finally, detection head and non-max suppression provide the output as bounding boxes. The proposed scheme was tested on three publicly available datasets, NPS, FLDrones, and AOT, and performance is computed in terms of AP, precision, and recall. The average precision (AP) on all the datasets (NPS-0.95, FLDrones-0.75, and AOT-0.80) shows its effectiveness compared to the existing techniques. It was also found from the results that by increasing the input resolution, the detection performance also improved. Zheng et al. [6] presented a new dataset that contains more than 13000 images of micro-UAVs acquired by another UAV and named it DetFly. This dataset was obtained under four environmental backgrounds: sky, urban, field, and mountains. It covers various scenarios, including viewing angles, background scenes, relative ranges, and lighting conditions. Then for detection purposes, they implemented eight different deep learning models, namely, Cascade R-CNN, FPN, Faster R-CNN, Grid R-CNN, RefineDet, RetinaNet, SSD512, and YOLOv3. With the IoU threshold of 0.5, the average precision (AP) of all the models was computed, and it was found that the Grid R-CNN has the highest average precision of 82.4%.

You only look once (YOLO) is a convolutional neural network-based model that has gained recent popularity because of its fast and precise performance. Gonzalez et al. [28] used YOLOv3 tiny for short and long-range-UAV detection while dadboud et al. [29] proposed a YOLOv5-based air-to-air object detection. They also tested their model with the DetFly dataset and some other datasets of ground-to-air environments like multiple drones and drones versus birds. This architecture was compared with Faster R-CNN and FPN models, and it was found that the YOLOv5 achieved better detection results. Kizilay and Aydin [30] proposed a YOLO variant, YOLOR which utilizes the information of the lower layers and consists of both the open and implicit learning. While implicit learning relies on learning based on prediction and experience with deeper layers, open learning often involves completing the learning process over open data. Table 1 highlights different deep-learning algorithms for G2A and A2A UAV detection applications. Moreover, it also provides the details of the dataset opted for by different researchers with their environment and camera details. This information helps the dataset selection and provides a brief view of the current UAV detection models.

Table 1 Deep Learning algorithms for UAV detection

Ref	Dataset used	Camera Position	Environment	Type (G2A / A2A)	Deep Learning Algorithm
Jamil et al. [15]	Customized Dataset	Ground	Outdoor	G2A	ViT multi-layer perceptron
Dil et al. [16]	Publicly available dataset on Roboflow	Ground	N/G	G2A	MFNet
Javan et al. [17]	Publicly available video data	N/G	Outdoor	G2A	YOLOv4
Zhao et al. [18]	DUT Anti-UAV	Ground	Outdoor	G2A	Faster-RCNN, Cascade-RCNN, ATSS, YOLOX, SSD
Kabir et al. [19]	Anti-UAV Amateur UAV	Ground	Outdoor	G2A	YOLOv4, YOLOv5, DETR
Leong et al. [23]	Own Dataset	Onboard	Outdoor	A2A	YOLOv3-tiny
Shinde et al. [31]	Custom dataset in Gazebo	NA	NA	G2A	YOLOv2
Carrio et al. [32]	Custom dataset using AirSim	Onboard	Indoor and Outdoor	NA	YOLOv2-tiny
H. Liu et al. [33]	Own Dataset	Ground	Outdoor	G2A	YOLOv2
Srigraroom et al. [34]	Own dataset	Ground	Outdoor	G2A	YOLOv3
Unlu et al. [35]	Own dataset	Ground	Outdoor	G2A	YOLOv3
Seidaliyeva et al. [5]	Birds' vs Drone	Ground	Outdoor	G2A	MobileNetv2
Sun et al. [24]	Own Dataset	Ground	Outdoor	G2A	TIB-Net
Abbasi et al. [25]	Google Images	Ground	Outdoor	G2A	ResNet50 with SVM
Schilling et al. [26]	Custom dataset in Gazebo	N/G	N/G	N/G	Improved CNN
Sangam et al. [27]	NPS-Drones FL-Drones AOT	Onboard	Outdoor	A2A	TransVisDrone
Zheng et al. [6]	DetFly	Onboard	Outdoor	A2A	Grid RCNN, SSD512, YOLOv3
Gonzalez et al. [28]	Own dataset	Onboard	Outdoor	A2A	YOLOv3
Dadboud et al. [29]	Multiple Datasets	Ground and Onboard	Outdoor	Both G2A and A2A	Faster RCNN, YOLOv5
Kizilay & Aydin [30]	DetFly	Onboard	Outdoor	A2A	YOLOR, YOLOv5, YOLOv4

3. Object Detection using YOLO based Algorithms

Object detection is one of the important tasks in computer vision, and the task of detecting a UAV in the aerial background is one of the detection problems. It aims at locating a region of interest inside an image and categorizing it as what a standard image classifier would

do. However, several points of interest referring to different or similar things might be present in a single image which elevates the object detection to a more complex classification issue. The two primary types of object detection algorithms are single stage and two-stage detectors [36]. The single-stage object detectors treated the object detection problem as a regression problem and do not need a region proposal stage that potentially consists of objects. It helps them to be faster than the two-stage detectors such as R-CNN, Fast RCNN, Faster RCNN, etc. The popularity of the single-stage detector You Only Look Once (YOLO) can be ascribed to its efficiency and precision. YOLO is a single stage detector that detects the existence of an object in a grid and provides information about the object such as its probability, bounding box coordinates, and class(es). Some of the issues in the base version of YOLO architecture are: (i) overlapping of multiple bounding boxes, and (ii) inability to detect center of two objects lying in same grid. The main building blocks of YOLO architecture are its backbone, neck, and head. The backbone carries out the feature extraction and pooling process, the neck aggregates low-level physical features to build feature maps, and the head consists of several convolutional layers that govern the output. This section describes the evolution of different YOLO models based on their architectures, from YOLO, YOLOv2 to YOLOv7.

YOLOv1: It is the first version of the YOLO model which uses a GoogleNet based backbone with 24 Convolutional layers and 2 fully connected layers. It is the first enhanced version of the YOLO model which increases the input resolution from 224 to 448 for testing but has several limitations. The first limitation is that this model can predict only two boxes in each grid which belong to the same class. Moreover, this model faces a lot of issues for small objects, and objects in new or unusual configurations [37].

YOLOv2: In this YOLO version, Darknet19 is used as the backbone and utilizes anchor boxes to predict the bounding boxes. This model down-samples the input randomly and changes the dimensions for every 10 batches. The smallest input image size is 320 x 320 and largest is 608 x 608. It also provides the fine-grained features that improve the performance of the model in terms of detection [38].

YOLOv3: This is the third version of the YOLO-based object detection architecture that aims at improving the speed and accuracy of past YOLO versions [39]. The primary factor in YOLOv3 is its backbone that uses Darknet53, a variant of a ResNet architecture with 53 convolution layers. Anchor boxes of various sizes and aspect ratios were another addition to YOLO v3 to detect different-sized objects as compared to same size anchor boxes in YOLOv2. Moreover, this pipeline also introduced the Feature Pyramid Networks (FPN) which enhances low level features for multi-scale object detection. The bounding box (BBox) prediction in YOLOv3 is the same as YOLOv2, detecting four coordinates for each BBox. In addition, it utilizes the sum of square error loss during and binary cross entropy loss function during training for class prediction. Moreover, it predicts boxes at three distinct sizes and uses a feature extraction method equivalent to feature pyramid networks [40] to extract the features. Finally, it added multiple convolutional layers to the feature extractor, with the final layer predicting a 3-D tensor encoding bounding box, objectness score, and class predictions. YOLOv3 has been widely opted by researchers for different applications of object detection.

Moreover, different improved versions of YOLOv3 which consider detection accuracy, speed, size of the model, etc. Like, it was enhanced with a spatial pyramid pooling mechanism and by adding a prediction layer for the application of vehicle detection [41]. This improvement adds the capability of multi-scaling and hence improves detection accuracy. Similar improvements were made for ship detection [42] and tomato detection with dense architecture and different activation functions [43]. The other improved versions are Mini-YOLOv3 which reduces the model size by 23%, trainable parameters, and floating-point operations, which reduces the time by nearly half [44], tiny YOLOv3 [45], clustering and Markov chain-based improvement to speed up the prediction of BBox [46], network pruning-based YOLOv3 for pedestrian detection [47], and for sheep face detection [48]. In addition to that, researchers also replaced the backbone of YOLOv3 to improve its accuracy and added attention modules [49]. For instance, the new backbones were MobileNetV2, used for surface defect detection [50], SqueezeNet for bridge surface detection [51], and a combination of CSPNet and Darknet53 for safety helmet detection [52].

YOLOv4: This version of the YOLO algorithm was developed to increase the training speed for the object detectors and its optimization for parallel computations. The concept of Bag-of-Freebies (BoF) and Bag-of-Specials (BoS) were added to YOLOv4 [53]. Without extending the inference time, the BoF increases the detector's accuracy and reduces the training cost. Conversely, the BoS slightly raises the inference cost while considerably enhancing object detection accuracy. In BoF, the problem of data imbalance was resolved using data augmentation methods like Mosaic, Self-Adersarial Training (SAT), and DropBlock regularization with class label smoothing was also added to the backbone. On the other hand, BoS is the post-processing method that improves the model's accuracy. It comprises mish activation, multi-input weighted residual connections, and cross-stage partial connection. Like YOLOv3, YOLOv4 is also used for several different applications of object detection and has some improved versions for different applications.

An improved version of a YOLOv4 backbone with information fusion in the neck and adaptive scaling was proposed for face mask detection in [54]. In addition, the backbone was also enhanced by combining it with DenseNet and a circular marker box instead of a rectangular BBox to detect cherry fruit with higher accuracy [55]. YOLOv4 with DenseNet were used for uneaten feed pellets detection in underwater images [56] and agriculture growth state [57]. Besides this, several other improvements based on pruning [58], was developed for object detection applications. Along with these modifications, YOLOv4 was also modified by compressing the model to make a tiny YOLOv4 with different improvements, which reduces the time and space complexities and improves the performance of the YOLOv4 [59]. In addition to that, the modifications were made using the attention mechanisms like ultra-lightweight subspace attention mechanism (ULSAM) [60], convolutional block attention module (CBAM) [61]. By

replacing backbones, for instance with EfficientNet [62], MobileNetv2 with Ghost module [63], MobileNet v2 with depth-wise separable convolution [64], and some minor modifications using Non-max suppression (NMS) [65], Reverse Depth wise Separable Convolution (RDSC) [66], Deep SORT [67], sliding slicing and discarding edges [68], and multi-channel fusion [69]were carried out.

YOLOv5: The architecture of YOLOv5 is similar to YOLOv4 and is improved by adding the Logit loss, which is better than binary cross entropy loss and focal loss function. Moreover, its neck uses only PANet. Like the other two versions of YOLO, YOLOv5 was also utilized for several applications and has different modified versions to improve accuracy, detection, speed etc. depending on its application requirement. This model was improved for several applications to improve small-scale object detection, accuracy, time, etc., based on different mechanisms. The first main improvement was made by replacing or adding module architectures in the backbone, for instance, the Ghost bottleNet algorithm [70], Involution Bottleneck [71], Ghost bottleneck [72], CoordConv [73], and BottleneckCSP-2 [74]. In addition, attention mechanism-based improvement were also made for YOLOv5, which utilizes CBAM [75] and squeeze-and-excitation (SE) module [76] Moreover, transformer-based mechanism [77] lip-mosaic algorithm [78], ghost architecture [79], data cleaning and augmentation[80] , multi-scaling [81], multi-level features [82] etc. modules were designed to improve YOLOv5 performance.

YOLOv6: The recent version of YOLO, YOLOv6 developed explicitly for fast processing in real-time object detection [83]. It uses RepVGG-based architecture in the backbone where RepBlocks are the main building blocks for small network models and CSPStackRepBlocks for large network models. The Path Aggregation Network (PAN) PANet topology is used in the neck, enhanced with RepBlocks or CSPStackRepBlocks in the backbone, also known as RepPAN. The final detection is done on the head, and in YOLOv6, an efficient decoupled head is used. Unlike the previous versions of YOLO, YOLOv6 is not yet fully utilized for different application areas and has limited applications. Moreover, improvements in YOLOv6 were also minimal; for instance, the neck of YOLOv6 was improved with a bidirectional concatenation for accurate localization [84], and Bep(Beer-mug) unit with BepC3(or named CSPStackRep) block [85] to improve the performance of YOLOv6.

YOLOv7: This version of YOLO beats all other versions both in terms of speed and accuracy. This architecture is concatenation-based network and has a new scaling method. This version utilizes Extended-ELAN as a backbone of the network which enhances the learning ability of the model by using shuffle, expand, and merge cardinality. Moreover, it changes the computation blocks only and uses group convolutions to expand the cardinality and channels of the computational blocks. Furthermore, it utilized compound scaling methods with both depth and width factors and computed the changes. Other than this, the model is a re-parameterized model with residual and concatenation connections using gradient flow propagation paths. There are two heads in the model, (i) auxiliary head which assists the training and (ii) lead head which is responsible for final output. YOLOv7 was also improved based on attention-based models [86] like CBAM , [87], Global Attention Mechanism [88],the Coordinate Attention (CoordAtt) module [89], and hybrid attention ACmix [90]. Moreover, NMS based methods [91], and image defogging module IDOD [92] based techniques were used to improve the architecture based on the application.

4. Material and Methods

The primary focus of this research work is to implement the YOLO based architecture for A2A UAV detection and analyze its performance on different YOLO models as a one class classification. Different YOLO architectures including, YOLOv3, YOLOv4, YOLOv5, YOLOv6, and YOLOv7 are trained with the DetFly dataset and compared using different performance metrics. Figure 1 highlights the main building blocks of the YOLO based models and is represented in Fig. 1. This section further describes different aspects of all the YOLO models along with their activation functions, loss functions, and training parameters.

4.1 Model Architecture

Convolutional Neural Networks (CNN) are classification models that can evaluate incoming images as ordered arrays of data and spot patterns in them. YOLO is CNN architecture that is quicker than other networks with good accuracy. Since it enables the model to view the entire image during testing, its predictions are influenced by the image's overall context. YOLO defines a confidence score for the areas according to their resemblance with the predetermined class and this score is used to decide whether an object is detected positively or not. The three main building blocks of the YOLO architecture are: backbone, neck, and head. The backbone extracts the features, the neck aggregates it and the head provide the final detection class. The main difference between these three sub-models' models are their methods and architectures as shown in Fig. 2 and explained in detail in the following sub-sections.

4.1.1 Backbone: The backbone in YOLO consists of several convolutional layers that extract the important features from the given input images and perform feature selection using pooling operations. The kernel size of the pooling operation defines the window size that moves around the whole image to extract features, and stride defines the step size for the window. The pooling operation is of two types: (a) MaxPool- selects the maximum, and AvgPool- computes the average value. The backbone architecture of different YOLO models is presented in Fig. 3.

- YOLOv3 uses DarkNet-53 as its backbone which has both 3 x 3 and 1 x 1 kernel size as convolution layers with different number of filters and uses AvgPool globally. The utilization of Darknet-53 as backbone makes the detection more accurate and faster.
- YOLOv4 and YOLOv5 use CSP-DarkNet with spatial pyramid pooling (SPP) as its backbone. Here, CSP means cross stage partial networks which divides the input into two halves in every layer of feature extraction and concatenates the output of convolution

of the first half to the second half of the original image directly.

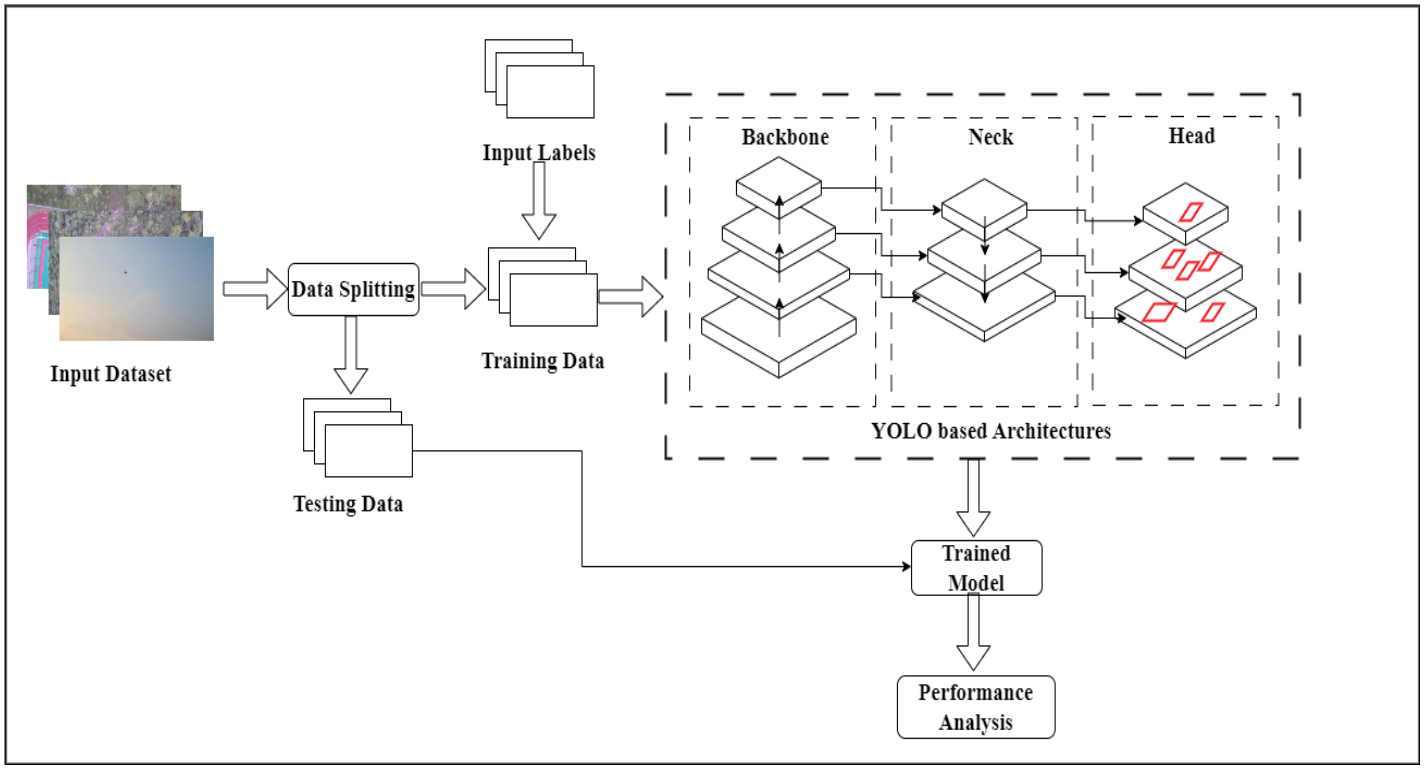


Fig. 1. Model architecture for comparing different YOLO versions.

Architecture	YOLOv3 (2018)	YOLOv4 (2020)	YOLOv5 (2021)	YOLOv6 (2022)	YOLOv7 (2022)
Backbone	Darknet53	CSPDarkNet	CSPDarkNet	RepVGG and CSPRepStack	E-ELAN
Neck	FPN	SPP with PAN	PANet	RepPAN	Compound Scaling based concatenation
Head	B x (S x S x (5 + C))	B x (S x S x (5 + C))	B x (S x S x (5 + C))	Decoupled Classification and Detection Head	auxiliary and lead head

Fig.2. Different YOLO architectures

- In YOLOv6, an EfficientRep is used as a backbone based on RepVGG. RepVGG is a structural re-parametrization method used to decouple the training time for multi-branch topology with an inference time plain architecture to achieve better accuracy. This design of efficient re-parameterization backbone is known as EfficientRep. Further in this, where RepBlocks are the main components during the training phase for small models, and each RepBlock is converted to a 3 x 3 convolution layer with a ReLU activation function.
- YOLOv7 proposed an Extended-ELAN (E-ELAN) which is based on the efficient layer aggregation network (ELAN). The ELAN is a deeper learning network which requires the control of the gradient path parameters for faster convergence. The E-ELAN utilizes expand, shuffle, and merge cardinality to improve the learning ability of the model and modifies the computational block only.

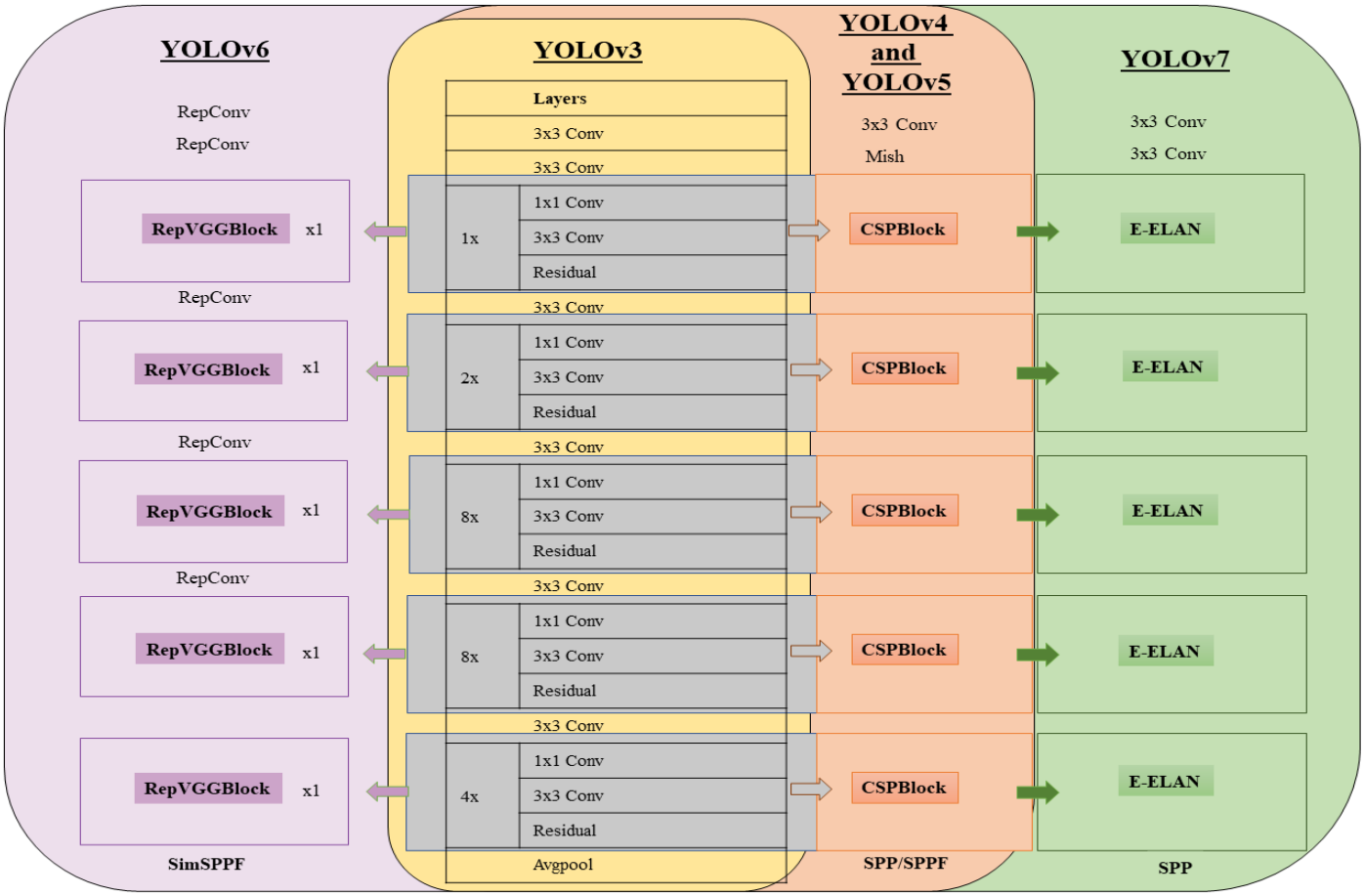


Fig.3. Overview of backbone in different YOLO models

4.1.2 Neck: The main function of the neck in YOLO based architectures is to aggregate the feature maps generated by backbone and pass them to the next phase for prediction or classification. This directly helps in dealing with the multi-scale objects (or objects in different scales) in the images. YOLO architectures use different networks for its neck including Feature Pyramid Network (FPN), Path Aggregation Network (PANet), modified PAN (RepPAN), etc.

- YOLOv3 uses FPN for its neck where FPN combines the low-level features with the high-level features to generate rich semantics with top-down and bottom-up pathways.
- YOLOv4 utilizes a modified path aggregation network (PANet) where a spatial pyramid pooling (SPP) block is used to extract important features and concatenates it with the original path aggregation network (PAN) for feature aggregation.
- YOLOv5 improved this neck architecture by using a variant of SPP, called SPPF to improve its speed, and modified PANet with CSPNet to improve the performance.
- YOLOv6 also uses the modified PAN, called RepPAN and concatenates features from different reparametrize blocks.
- YOLOv7 opted for a compound scaling-based concatenation model for depth and width factor, the properties of the model.

4.1.3 Head: The final stage of YOLO models is a detection/prediction head that consists of three branches that predict class probabilities, objectness score, and bounding boxes. In order to forecast the final prediction, the YOLOv3 detection head consists of three convolutional layers that down sample the feature map produced by the backbone network. Each of these layers combines the feature maps from the preceding layer to obtain the final predictions using a 1x1 convolution. A grid of cells that predict the existence of objects of various sizes and their locations is the output of the YOLOv3 detection head and provides a tensor of the structure B x (S x S x (5 + C)). Here, B is the batch size, C is the number of object classes to be detected, and S is the size of the output feature map. In the case of YOLOv3, YOLOv4 and YOLOv5 this size is the same and differs only in their structure. In YOLOv4, the detection head additionally employs anchor boxes which are pre-defined boxes of various sizes and aspect ratios. YOLOv5 detection head is improved with CSP architecture that provides efficient features and better resources while being anchor free. The YOLOv3, YOLOv4, YOLOv5 architecture have a coupled head that shares the parameters between the classification and localization. Unlike these, YOLOv6 has an efficient decoupled head that uses a hybrid channel strategy, wherein the classification and localization is separate and requires additional two convolution layers size 3x3. The width multiplier in YOLOv6 scales the neck's width for both the backbone and neck, thereby reducing the computational cost and inference latency. YOLOv7 also separates the

auxiliary and lead head and assigns using their own predictions with ground truth values. The lead head predictions are used as a guidance to generate coarse-to-fine hierarchical labels that are further used for the learning of auxiliary head and lead head. In this way, the labels are dynamically adjusted during learning and makes the final prediction more accurate.

In the development and training of deep learning models, activation and loss functions are both essential elements. Since most real-world data is non-linear in nature, activation functions are used to bring non-linearity into the model. Loss functions, on the other hand, reflect the effectiveness of a model in handling the problem it was designed to address. Selection of an appropriate activation and loss function is an important factor and has been discussed in the following subsection for different YOLO models.

4.2 Activation Functions

The activation functions which are commonly used by different deep learning models, including sigmoid, ReLU, Tanh, SoftMax, swish, LeakyReLU, ELU, etc [93]. Each function has its own pros and cons depending on the model. YOLOv3 uses LeakyReLU, an advanced version of ReLU activation function, that helps in avoiding the dying ReLU problem. YOLOv4 uses two different activation functions - Mish [94] and ReLU. combination helps the model in balancing its accuracy with computational efficiency. YOLOv5 utilizes SiLU activation function which utilizes sigmoid function and performs better than ReLU activation function. It also utilizes swish in some of its layers to improve computational efficiency. Similar to YOLOv5 and YOLOv6, YOLOv7 also utilizes SiLU activation functions yielding accurate results.

Table 2 Activation and Loss Functions

		YOLOv3	YOLOv4	YOLOv5	YOLOv6	YOLOv7
Activation Function		Leaky ReLU	Mish and ReLu	SiLU	SiLU	SiLU
Loss Function	**Localization Loss**	MSE	IoU	IoU	DFL, GioU and SioU	IoU
	Object Loss	BCE	BCE	BCE	VFL	BCE
	Classification Loss	BCE	BCE + Focal loss	BCE + Label Smoothing	BCE	BCE + Focal Loss

4.3 Loss Functions

It is a mathematical function that uses actual and predicted values to compute the loss for the actual predictions which is minimized for achieving better results. There are several loss functions that are used in YOLO such as Mean Squared Error (MSE), Binary Cross-Entropy (BCE), Categorical Cross-Entropy (CCE), Mean Absolute Error (MAE), Huber Loss, etc. [95]. These loss functions can be of three types: (a) Localization Loss, (b) Object Loss, and (c) Classification Loss. Localization loss measures the loss or error in bounding box coordinates, object loss is a measure of objectness score, and classification loss computes the class probability loss. In YOLOv3, MSE is used for localization loss, BCE for both the object and classification loss. YOLOv4, YOLOv5, and YOLOv7 utilized an IoU loss function whereas it uses BCE for both object and classification loss. YOLOv4 and YOLOv7 added focal loss for classification loss in order to address the class imbalance problem. On the other hand, YOLOv5 added label smoothing loss for classification loss to prevent the model from becoming too confident in its predictions. Furthermore, YOLOv6 uses distribution focal loss (DFL) with IoU loss function, GioU and SioU for localization loss, Varifocal loss (VFL) for classification loss, and BCE for object loss.

4.4 Training parameters

The training parameters in YOLO-based models include batch size, learning rate, momentum, weight decay, input size, number of epochs, and optimizer [96]. These parameters affect the speed and accuracy of training and must be tuned for specific applications and dataset. The details of these parameters are described as follows:

- Batch size refers to the number of images that are processed together in each training iteration. A larger batch size can result in faster training times as more images are processed in parallel at the cost of increased memory requirements.
- The step size used to update the model weights during each iteration of training is determined by the learning Rate, a critical hyperparameter in neural network training. Since the model weights are updated more often, a higher learning rate can speed up the convergence, but can also cause the optimizer to overshoot the ideal solution, leading to unstable or erroneous training. On the other side, a slower learning rate may hinder convergence, but could also enable the optimizer to adjust the weights precisely.
- Momentum is a hyperparameter used in optimization algorithms that smooths out the updates to the model weights during training. The smoothing parameter controls the update rate of the weights and helps in stabilizing and accelerating the optimization process.
- Weight decay is a regularization technique used to prevent overfitting. It adds a penalty term to the loss function that encourages the model weights to stay small. The regularization term is typically the L2 norm of the weights, and the weight decay hyperparameter controls the strength of the regularization term.
- Input Size is the size of the image which is passed during training. The larger the input size the higher is

the performance accuracy and so is the memory requirement.

- Number of epochs refers to the many times the full training dataset is processed. A higher epoch size will lead to better convergence and accuracy.
- Optimization is used to update the model weights during training where stochastic gradient descent with momentum (SGDM) or adaptive moment estimation (Adam) are commonly used optimizers for YOLO based models [97].

5. Experimentation and Analysis

The performance of all the YOLO-based architectures is quantified in terms of their accuracy against the ground truth. The experimentation parameters are detailed in the following subsection.

5.1 Environment Setup

The experimental setup utilized an NVIDIA RTX graphics card with 20 GB memory, and the other hardware specifications as listed in Table 3. The experiment was conducted on the VS Code framework, with Ubuntu 20.04 as the operating system.

Table 3 System Specification for proposed work

Hardware	Specification
Computer	GPU
CPU	Intel Xeon E5 (3.7 GHz)
RAM	48GB (6 x 8GB)
GPU	RTX A4500 (20GB)

5.2 Dataset used

Among the few datasets available in the public domain for A2A UAV detection, DeTFly dataset is a large-scale image dataset of MicroUAVs (small objects) which is used for training and testing of different YOLO architectures. Zheng et al. [6] published this dataset (DeTFly) in the year 2021 and made it available in the public domain for research purposes. They also tested this data for UAV detection using different deep learning models and achieved a 90.1% average precision (maximum) using Grid R-CNN. This dataset includes 13271 images of the target micro-UAV taken using DJI Mavic. Each image is of size 3840 x 2160 pixels and contains a manually annotated dataset that experts have compiled. It also comprises images in different lighting conditions, background sceneries, relative distances, and viewing angles and are divided as: the sky, the urban, the field, and the mountain in the background data.

Sky	Urban	Field	Mountain

Fig. 4. Dataset Samples

Fig. 4 shows sample images from the dataset in different environmental backgrounds. The dataset has an equal proportion (20 - 30%) of the samples for these backgrounds with front, top, and bottom view of the UAV. The image samples are captured in different lighting conditions from morning to evening. This dataset has a 10.8% sample with strong/ weak lighting conditions, 11.2% with motion blur, and 0.8% with partial target occlusion.

5.3 Training Parameters

In this work, YOLO models are trained with the DeTFly dataset, where 70% of data samples are used for training of which 10% data is used for validation and 30% for testing. The training parameters for all the tested YOLO models are kept mostly the same to benchmark their performance and is shown in Table 4.

Table 4 Training parameters

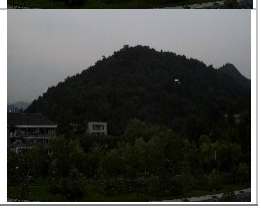
Parameter	Values				
	YOLOv3	YOLOv4	YOLOv5	YOLOv6	YOLOv7
Input Size	640	416	640	640	640
Batch Size	16	32	16	16	16
Optimizer	SGD	SGD	SGD	SGD	SGD
Learning rate	1e-2	1e-2	1e-2	1e-2	1e-2
Momentum	0.93	0.93	0.93	0.93	0.93
Weight Decay	5e-4	5e-4	5e-4	5e-4	5e-4
Epoch*	300	10000*	300	300	300

*represents the iterations and not the epochs.

5.4 Detection Results

The visual results of the detection using all the models on the image samples from different environments are shown in table 5. The visual results presented the bounding box on the UAVs and gave their confidence score. Confidence score is an evaluation standard that computes the probability for correct detection results. The higher the probability means more chances that the object belongs to the class.

Table 5 Detection Results using different YOLO models.

Models	Detection Results (Different + Environments)			
	Sky	Urban	Field	Mountain
YOLOv3				
YOLOv4				
YOLOv5				
YOLOv6				
YOLOv7				

The confidence score to detect a UAV in the above images is more than 50% and the same is with the complete test data. These results also reflect that detection is also possible in complex environments and different backgrounds using these models.

5.5 Experimentation Results

These models are quantitatively benchmarked using different performance metrics including mean average precision (mAP@0.5 and mAP@0.5:0.95). Here mAP@0.5 means average precision with Intersection over Union (IoU) threshold 0.5. IoU is the ratio of area overlapped between the predicted box and its ground truth. On the other hand, mAP@0.5:0.95 takes the average values with the IoU thresholds from 0.5 to 0.95 with step size 0.05 by default.

The model showed that the performance of YOLOv5 in terms of both mAP@0.5 and mAP@0.5:0.95 is better than all other models. Though previous literature claims higher mAP for YOLOv6 and YOLOv7 for other object detection problems, in this paper the number of epochs considered is only 300 and the model convergence is slower due to its complexity. The accuracy for YOLOv6 and YOLOv7 may have degraded because of several reasons including single class, limited data, complex backgrounds, and small objects in this specific task of UAV detection. The comparison based on the size of model is depicted in Fig. 5

Table 6 Performance of YOLO algorithms

Performance Metric	YOLOv3	YOLOv4	YOLOv5	YOLOv6	YOLOv7
mAP@0.5	0.9756	0.8833	0.9784	0.90487	0.893
mAP@0.5:0.95	0.7044	0.2901	0.7231	0.62746	0.548

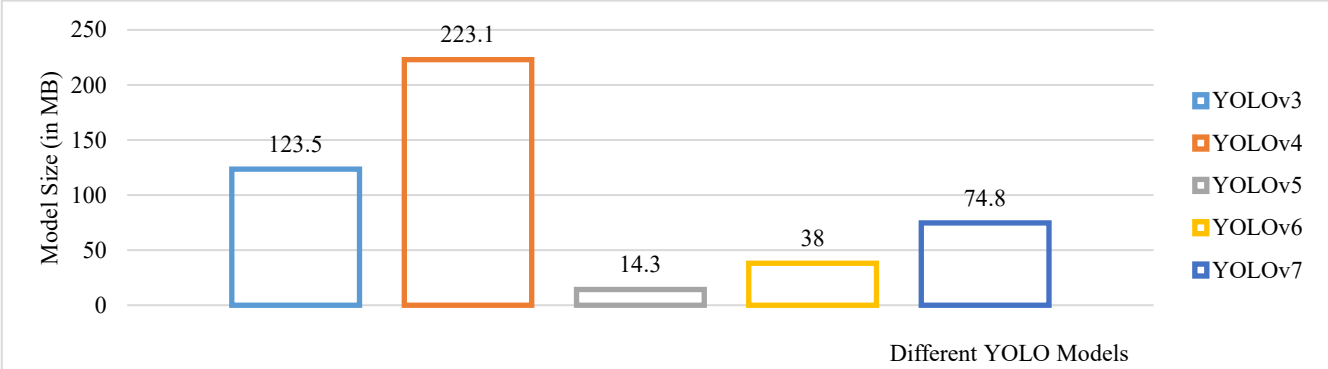


Fig. 5. Comparison of YOLO architectures based on model size

The comparison based on model size shows that YOLOv5 is the smallest model among all other models as it has simple, multi-branch and partial network architecture. Although YOLOv4 also has a similar backbone, operations like Bag of Freebies and Bag of Specials make it complex. YOLOv6 and YOLOv7 are recent architecture with large model size since there are no partial connections in YOLOv6 and YOLOv7 have complex E-ELAN module.

5.6 Comparison with state of art

The DeTFly dataset was developed for the application of A2A micro-UAV detection where both the UAVs are in the air with different angles, views, environmental conditions and moving with different speed and height. This dataset has been limitedly explored for the detection of UAVs using different deep learning strategies as shown in table 7.

Table 7 DeTFly and Deep Learning Models

Deep Learning Models	Faster R-CNN	Grid R-CNN	Cascade R-CNN	YOLOv3	YOLOv4	YOLOv5	YOLOv6	YOLOv7
DetFly	✓	✓	✓	✓	✓	✓	×	×

Table 7 shows that the YOLOv6 has not been used for the detection of UAV and has also not been applied in the DetFly dataset. In this work, both the YOLOv6 and YOLOv7 models are implemented, and their performance is benchmarked with other schemes based on AP or mAP.

Table 8 Comparison based on AP/mAP.

Learning Architecture	Results from existing state of the art						Results from experimentation carried out				
	Faster R-CNN[6]	Grid R-CNN [6]	Cascade R-CNN[6] v	YOLO v3[6]	YOLO v4 [30]	YOLOv5 [29]	YOLOv3	YOLOv4	YOLO v5	YOLO v6	YOLO v7
AP	70.5	82.4	79.4	72.3	-	-	98.23	91.0	97.21	-	92.35
mAP	-	-	-	-	0.908	0.98	0.9756	0.8833	0.9784	0.90487	0.893

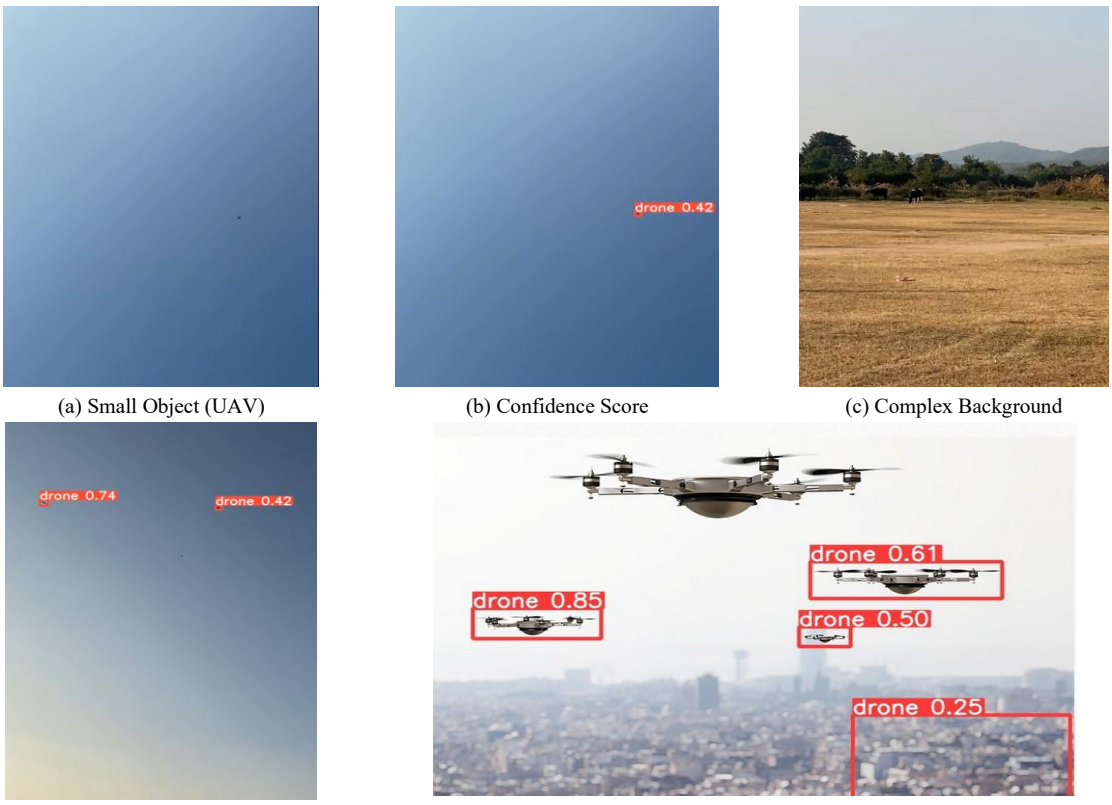
From the above comparison, the following conclusions can be made:

- The performance of single stage detectors i.e., YOLO based models is better than two stage detectors in terms of AP.
- YOLOv5 models have different outcomes, like the mAP of YOLOv5 from existing research articles is better than the YOLOv5 scheme implemented here. This is because the researchers in [30] have used an additional dataset to create a large-scale data (upto 1,00,000 samples) and train their model with that data.
- The YOLOv3 developed here has higher AP than the YOLOv3 due to parameter tuning.
- The performance of the detection can be improved with larger data and can be added to get better detection results.
- Though YOLOv6 and YOLOv7 are advanced models, their performance shows that there is still scope for improvement in them.

5.7 Challenges and Future Recommendations

The trained models discussed in the previous sections are also tested on real-time UAV video data and a UAV image dataset from Kaggle [98] to analyze their performances. The analysis shown in figure 6 presented the main challenges on where researchers need to focus. The details of these challenges are as follows:

- *Small-object detection*: This is the main challenge in these YOLO models that sometimes do not detect small size objects. Like in Fig. 6 (a), UAV is very far from the camera and the area covered by the object in the image frame of size 720 x 1280 is around 15 x 11 pixels. It was found that the UAV is not detected in this frame due to its small size. The small objects, due to their fewer pixel coverage, do not provide sufficient features to the learning model and is therefore not detected during testing. There is a wide scope for enhancement that will help in improving the performance of the detection system, especially in the case of small-scale objects.
- *Confidence Score*: Fig. 6(b) presents a case where a small-sized UAV is detected, but the confidence score is less than 50%, and in most of the real-time systems, the bounding boxes with a confidence score less than 50% are treated as a false detection. So, by improving the small-scale object detection, this can also be improved.
- *Complex Background*: The other most common challenge in the object detection problem is the complex background as shown in Fig. 6(c). In this case, the UAV with the complex background such as tree or sun where the UAV is barely visible with naked eye also becomes the problem for the detection system to detect accurately. This is also one of the future scopes where researchers can aim at improving detection accuracy.
- *UAV vs. Bird:* In Fig. 6(d), as per labels, two drones are detected, but, in actual cases, the object detected with a high confidence score is a bird, and on the other side, the drone is detected with a low confidence score. Though the model is not trained with the dataset of birds, false detection with high confidence score shows its limitations. So, in this case, two different pathways can be opted, where the first might go towards the classification of bird vs. drone and the second to avoid these kinds of false detections in a UAV detection model.



(a) Small Object (UAV) (b) Confidence Score (c) Complex Background

(d) Bird as Drone (e) Multi-scaling Issue

Fig. 6. Challenges during real-time video testing

- *Multi-scaling:* As these models are trained with the micro-UAV dataset, it does not detect the UAV, which is closer and larger, as presented in Fig. 6(e). So, multi-scaling is still a problem in these YOLO-based models that need the attention of the researchers.

6. Conclusion

In this paper, YOLO based different architectures have been experimented for the task of Air-to-Air UAV detection and has been shown to perform well for most of the cases. The main novelty of this work is a common benchmarking of different YOLO frameworks under similar training parameters and the testing of YOLO of newer architectures such as YOLOv6 and YOLOv7. It is found that the performance of YOLOv5 is better than other YOLO versions in terms of mAP and model size. YOLOv6 and YOLOv7 is designed especially for multi-class problems and are complex in nature as compared to YOLOv5 and could be one of the major reasons for their inability to converge under similar number of iterations. This paper does not explore individual improvement in each of these architectures and it is still an open research problem to module the training

algorithms for specific problems at hand. There is also a further scope of improvement in detection accuracy of UAV in these datasets or others for mAP@0.5:0.95.

Declarations

Funding This work has received funding from TiHAN-IITH under grant number: TiHAN-IITH/03/2022-23/196.

Acknowledgment The authors would like to thank TiHAN-IITH for their support through project funding.

Ethical Approval The work does not include any personal data relatable to identifiable living persons.

Conflicts of Interests The authors declare no conflicts of interest.

Availability of Data and Materials The dataset used for training is publicly available dataset and the data including some test samples, codes, etc. can be made available on reasonable request.

Authors Contribution

Dapinder Kaur: Conceptualization, Implementation, Data Collection, Writing- Original draft

Neeraj Battish: Implementation, Data Collection

Arnav Bhavsar: Writing- review and editing

Shashi Poddar: Conceptualization, Writing- review and editing

References

[1] "Unmanned Aerial Vehicle (UAV) Market Share, Size, Trends - [2022-2027]." https://www.marketsandmarkets.com/Market-Reports/unmanned-aerial-vehicles-uav-market-662.html (accessed Jan. 06, 2023).

[2] M. D. Shear and M. S. Schmidt, "White House Drone Crash Described as a U.S. Worker's Drunken Lark - The New York Times." https://www.nytimes.com/2015/01/28/us/white-house-drone.html (accessed Jan. 06, 2023).

[3] D. Furfaro, L. Celona, and N. Musumeci, "Civilian drone crashes into Army helicopter," *NewYork Post*, 2017. https://nypost.com/2017/09/22/army-helicopter-hit-by-drone/ (accessed Aug. 18, 2022).

[4] E. Unlu, E. Zenou, N. Riviere, and P.-E. Dupouy, "Deep learning-based strategies for the detection and tracking of drones using several cameras," *IPSJ Trans. Comput. Vis. Appl.*, vol. 11, no. 1, p. 7, Dec. 2019, doi: 10.1186/s41074-019-0059-x.

[5] U. Seidaliyeva, D. Akhmetov, L. Ilipbayeva, and E. T. Matson, "Real-Time and Accurate Drone Detection in a Video with a Static Background," *Sensors*, vol. 20, no. 14, p. 3856, Jul. 2020, doi: 10.3390/s20143856.

[6] Y. Zheng, Z. Chen, D. Lv, Z. Li, Z. Lan, and S. Zhao, "Air-to-Air Visual Detection of Micro-UAVs: An Experimental Evaluation of Deep Learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1020–1027, Apr. 2021, doi: 10.1109/LRA.2021.3056059.

[7] J. Sadovskis and A. Aboltins, "Modern methods for UAV detection, classification, and tracking," in *2022 IEEE 63th International Scientific Conference on Power and Electrical Engineering of Riga Technical University (RTUCON)*, IEEE, Oct. 2022, pp. 1–7. doi: 10.1109/RTUCON56726.2022.9978860.

[8] A. J. Garcia, J. Min Lee, and D. S. Kim, "Anti-Drone System: A Visual-based Drone Detection using Neural Networks," in *2020 International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, Oct. 2020, pp. 559–561. doi: 10.1109/ICTC49870.2020.9289397.

[9] M. Pavliv, F. Schiano, C. Reardon, D. Floreano, and G. Loianno, "Tracking and Relative Localization of Drone Swarms With a Vision-Based Headset," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1455–1462, Apr. 2021, doi: 10.1109/LRA.2021.3051565.

[10] J. Dudczyk, R. Czyba, and K. Skrzypczyk, "Multi-Sensory Data Fusion in Terms of UAV Detection in 3D Space," *Sensors*, vol. 22, no. 12, p. 4323, Jun. 2022, doi: 10.3390/s22124323.

[11] Y. Mo, J. Huang, and G. Qian, "Deep Learning Approach to UAV Detection and Classification by Using Compressively Sensed RF Signal," *Sensors*, vol. 22, no. 8, p. 3072, Apr. 2022, doi: 10.3390/s22083072.

[12] W. M. Martins, R. G. Braga, A. C. B. Ramos, and F. Mora-Camino, "A Computer Vision Based Algorithm for Obstacle Avoidance," 2018, pp. 569–575. doi: 10.1007/978-3-319-77028-4_73.

[13] B. Demir *et al.*, "Real-time high-resolution omnidirectional imaging platform for drone detection and tracking," *J. Real-Time Image Process.*, vol. 17, no. 5, pp. 1625–1635, Oct. 2020, doi: 10.1007/s11554-019-00921-7.

[14] B. Taha and A. Shoufan, "Machine Learning-Based Drone Detection and Classification: State-of-the-Art in Research," *IEEE Access*, vol. 7, pp. 138669–138682, 2019, doi: 10.1109/ACCESS.2019.2942944.

[15] S. Jamil, M. S. Abbas, and A. M. Roy, "Distinguishing Malicious Drones Using Vision Transformer," *AI*, vol. 3, no. 2, pp. 260–273, Mar. 2022, doi: 10.3390/ai3020016.

[16] M. Dil, M. U. Khan, M. Z. Alam, F. A. Orakazi, Z. Kaleem, and C. Yuen, "SafeSpace MFNet: Precise and Efficient MultiFeature Drone Detection Network," Nov. 2022, [Online]. Available: http://arxiv.org/abs/2211.16785

[17] F. Dadrass Javan, F. Samadzadegan, M. Gholamshahi, and F. Ashatari Mahini, "A Modified YOLOv4 Deep Learning Network for Vision-Based UAV Recognition," *Drones*, vol. 6, no. 7, p. 160, Jun. 2022, doi: 10.3390/drones6070160.

[18] J. Zhao, J. Zhang, D. Li, and D. Wang, "Vision-Based Anti-UAV Detection and Tracking," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 25323–25334, Dec. 2022, doi: 10.1109/TITS.2022.3177627.

[19] M. S. Kabir, I. K. Ndukwe, and E. Z. S. Awan, "Deep Learning Inspired Vision based Frameworks for Drone Detection," in *2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*, IEEE, Jun. 2021, pp. 1–5. doi: 10.1109/ICECCE52056.2021.9514124.

[20] P. Zhang, G. Chen, Y. Li, and W. Dong, "Agile Formation Control of Drone Flocking Enhanced With Active Vision-Based Relative Localization," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6359–6366, Jul. 2022, doi: 10.1109/LRA.2022.3171096.

[21] D. D. C. Trapal, B. C. C. Leong, H. W. Ng, J. T. G. Zhong, S. Srigrarom, and T. H. Chan, "Improvement of Vision-based Drone Detection and Tracking by Removing Cluttered Background, Shadow and Water Reflection with Super Resolution," in *2021 6th International Conference on Control and Robotics Engineering (ICCRE)*, IEEE, Apr. 2021, pp. 162–168. doi: 10.1109/ICCRE51898.2021.9435671.

[22] Y. H. Lau, N. S. Jun Liang, S. X. Seah, and S. Srigrarom, "Sunlight Compensation for Vision Based Drone Detection," in *2021 21st International Conference on Control, Automation and Systems (ICCAS)*, IEEE, Oct. 2021, pp. 442–443. doi: 10.23919/ICCAS52745.2021.9649961.

[23] W. L. Leong *et al.*, "Vision-Based Sense and Avoid with Monocular Vision and Real-Time Object Detection for UAVs," in *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, IEEE, Jun. 2021, pp. 1345–1354. doi: 10.1109/ICUAS51884.2021.9476746.

[24] H. Sun, J. Yang, J. Shen, D. Liang, L. Ning-Zhong, and H. Zhou, "TIB-Net: Drone Detection Network With Tiny Iterative Backbone," *IEEE Access*, vol. 8, pp. 130697–130707, 2020, doi: 10.1109/ACCESS.2020.3009518.

[25] K. Abbasi *et al.*, "A Vision-Based Amateur Drone Detection Algorithm for Public Safety Applications," in *2019 UK/ China Emerging Technologies (UCET)*, IEEE, Aug. 2019, pp. 1–5. doi: 10.1109/UCET.2019.8881879.

[26] F. Schilling, J. Lecoeur, F. Schiano, and D. Floreano, "Learning Vision-based Cohesive Flight in Drone Swarms," Sep. 2018, [Online]. Available: http://arxiv.org/abs/1809.00543

[27] T. Sangam, I. R. Dave, W. Sultani, and M. Shah, "TransVisDrone: Spatio-Temporal Transformer for Vision-based Drone-to-Drone Detection in Aerial Videos," Oct. 2022.

[28] F. Gonzalez, R. Caballero, F. J. Perez-Grau, and A. Viguria, "Vision-based UAV Detection for Air-to-Air Neutralization," in *2021 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, IEEE, Oct. 2021, pp. 236–241. doi: 10.1109/SSRR53300.2021.9597861.

[29] F. Dadboud, V. Patel, V. Mehta, M. Bolic, and I. Mantegh, "Single-Stage UAV Detection and Classification with YOLOV5: Mosaic Data Augmentation and PANet," in *2021 17th IEEE International Conference*

on Advanced Video and Signal Based Surveillance (AVSS), IEEE, Nov. 2021, pp. 1–8. doi: 10.1109/AVSS52988.2021.9663841.

[30] E. Kizilay and I. Aydin, "A YOLOR Based Visual Detection of Amateur Drones," in *2022 International Conference on Decision Aid Sciences and Applications (DASA)*, IEEE, Mar. 2022, pp. 1446–1449. doi: 10.1109/DASA54658.2022.9765252.

[31] C. Shinde, R. Lima, and K. Das, "Multi-view Geometry and Deep Learning Based Drone Detection and Localization," in *2019 Fifth Indian Control Conference (ICC)*, IEEE, Jan. 2019, pp. 289–294. doi: 10.1109/INDIANCC.2019.8715593.

[32] A. Carrio, S. Vemprala, A. Ripoll, S. Saripalli, and P. Campoy, "Drone Detection Using Depth Maps," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2018, pp. 1034–1037. doi: 10.1109/IROS.2018.8593405.

[33] H. Liu, F. Qu, Y. Liu, W. Zhao, and Y. Chen, "A drone detection with aircraft classification based on a camera array," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 322, p. 052005, Mar. 2018, doi: 10.1088/1757-899X/322/5/052005.

[34] S. Srigrarom, S. M. Lee, M. Lee, F. Shaohui, and P. Ratsamee, "An Integrated Vision-based Detection-tracking-estimation System for Dynamic Localization of Small Aerial Vehicles," in *2020 5th International Conference on Control and Robotics Engineering (ICCRE)*, IEEE, Apr. 2020, pp. 152–158. doi: 10.1109/ICCRE49379.2020.9096259.

[35] E. Unlu, E. Zenou, N. Riviere, and P.-E. Dupouy, "Deep learning-based strategies for the detection and tracking of drones using several cameras," *IPSJ Trans. Comput. Vis. Appl.*, vol. 11, no. 1, p. 7, Dec. 2019, doi: 10.1186/s41074-019-0059-x.

[36] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," *Multimed. Tools Appl.*, vol. 82, no. 6, pp. 9243–9275, Mar. 2023, doi: 10.1007/s11042-022-13644-y.

[37] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," Jun. 2015.

[38] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," Dec. 2016, [Online]. Available: http://arxiv.org/abs/1612.08242

[39] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," Apr. 2018, [Online]. Available: http://arxiv.org/abs/1804.02767

[40] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," Dec. 2016, [Online]. Available: http://arxiv.org/abs/1612.03144

[41] K.-J. Kim, P.-K. Kim, Y.-S. Chung, and D.-H. Choi, "Performance Enhancement of YOLOv3 by Adding Prediction Layers with Spatial Pyramid Pooling for Vehicle Detection," in *2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, IEEE, Nov. 2018, pp. 1–6. doi: 10.1109/AVSS.2018.8639438.

[42] L. Chang, Y.-T. Chen, J.-H. Wang, and Y.-L. Chang, "Modified Yolov3 for Ship Detection with Visible and Infrared Images," *Electronics*, vol. 11, no. 5, p. 739, Feb. 2022, doi: 10.3390/electronics11050739.

[43] M. O. Lawal, "Tomato detection based on modified YOLOv3 framework," *Sci. Rep.*, vol. 11, no. 1, p. 1447, Jan. 2021, doi: 10.1038/s41598-021-81216-5.

[44] Q.-C. Mao, H.-M. Sun, Y.-B. Liu, and R.-S. Jia, "Mini-YOLOv3: Real-Time Object Detector for Embedded Applications," *IEEE Access*, vol. 7, pp. 133529–133538, 2019, doi: 10.1109/ACCESS.2019.2941547.

[45] Z. Yi, S. Yongliang, and Z. Jun, "An improved tiny-yolov3 pedestrian detection algorithm," *Optik (Stuttg).*, vol. 183, pp. 17–23, Apr. 2019, doi: 10.1016/j.ijleo.2019.02.038.

[46] L. Zhao and S. Li, "Object Detection Algorithm Based on Improved YOLOv3," *Electronics*, vol. 9, no. 3, p. 537, Mar. 2020, doi: 10.3390/electronics9030537.

[47] Y. Shao, X. Zhang, H. Chu, X. Zhang, D. Zhang, and Y. Rao, "AIR-YOLOv3: Aerial Infrared Pedestrian Detection via an Improved YOLOv3 with Network Pruning," *Appl. Sci.*, vol. 12, no. 7, p. 3627, Apr. 2022, doi: 10.3390/app12073627.

[48] S. Song *et al.*, "Using Pruning-Based YOLOv3 Deep Learning Algorithm for Accurate Detection of Sheep Face," *Animals*, vol. 12, no. 11, p. 1465, Jun. 2022, doi: 10.3390/ani12111465.

[49] K. Wang and M. Liu, "YOLOv3-MT: A YOLOv3 using multi-target tracking for vehicle visual detection," *Appl. Intell.*, vol. 52, no. 2, pp. 2070–2091, Jan. 2022, doi: 10.1007/s10489-021-02491-3.

[50] X. Chen, J. Lv, Y. Fang, and S. Du, "Online Detection of Surface Defects Based on Improved YOLOV3," *Sensors*, vol. 22, no. 3, p. 817, Jan. 2022, doi: 10.3390/s22030817.

[51] S. Teng, Z. Liu, and X. Li, "Improved YOLOv3-Based Bridge Surface Defect Detection by Combining High- and Low-Resolution Feature Images," *Buildings*, vol. 12, no. 8, p. 1225, Aug. 2022, doi: 10.3390/buildings12081225.

[52] L. Deng, H. Li, H. Liu, and J. Gu, "A lightweight YOLOv3 algorithm used for safety helmet detection," *Sci. Rep.*, vol. 12, no. 1, p. 10981, Jun. 2022, doi: 10.1038/s41598-022-15272-w.

[53] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," Apr. 2020, [Online]. Available: http://arxiv.org/abs/2004.10934

[54] J. Yu and W. Zhang, "Face Mask Wearing Detection Algorithm Based on Improved YOLO-v4," *Sensors*, vol. 21, no. 9, p. 3263, May 2021, doi: 10.3390/s21093263.

[55] R. Gai, N. Chen, and H. Yuan, "A detection algorithm for cherry fruits based on the improved YOLO-v4 model," *Neural Comput. Appl.*, May 2021, doi: 10.1007/s00521-021-06029-z.

[56] X. Hu *et al.*, "Real-time detection of uneaten feed pellets in underwater images for aquaculture using an improved YOLO-V4 network," *Comput. Electron. Agric.*, vol. 185, p. 106135, Jun. 2021, doi: 10.1016/j.compag.2021.106135.

[57] A. M. Roy and J. Bhaduri, "Real-time growth stage detection model for high degree of occultation using DenseNet-fused YOLOv4," *Comput. Electron. Agric.*, vol. 193, p. 106694, Feb. 2022, doi: 10.1016/j.compag.2022.106694.

[58] H. Liu, K. Fan, Q. Ouyang, and N. Li, "Real-Time Small Drones Detection Based on Pruned YOLOv4," *Sensors*, vol. 21, no. 10, p. 3374, May 2021, doi: 10.3390/s21103374.

[59] A. Kumar, A. Kalia, A. Sharma, and M. Kaushal, "A hybrid tiny YOLO v4-SPP module based improved face mask detection vision system," *J. Ambient Intell. Humaniz. Comput.*, Oct. 2021, doi: 10.1007/s12652-021-03541-x.

[60] L. Tan, X. Lv, X. Lian, and G. Wang, "YOLOv4_Drone: UAV image target detection based on an improved YOLOv4 algorithm," *Comput. Electr. Eng.*, vol. 93, p. 107261, Jul. 2021, doi: 10.1016/j.compeleceng.2021.107261.

[61] B. Yang, Z. Gao, Y. Gao, and Y. Zhu, "Rapid Detection and Counting of Wheat Ears in the Field Using YOLOv4 with Attention Module," *Agronomy*, vol. 11, no. 6, p. 1202, Jun. 2021, doi: 10.3390/agronomy11061202.

[62] L. Wu, J. Ma, Y. Zhao, and H. Liu, "Apple Detection in Complex Scene Using the Improved YOLOv4 Model," *Agronomy*, vol. 11, no. 3, p. 476, Mar. 2021, doi: 10.3390/agronomy11030476.

[63] D. Bu, S. Guo, and H. Li, "sEMG-Based Motion Recognition of Upper Limb Rehabilitation Using the Improved Yolo-v4 Algorithm," *Life*, vol. 12, no. 1, p. 64, Jan. 2022, doi: 10.3390/life12010064.

[64] M. Zhang, S. Xu, W. Song, Q. He, and Q. Wei, "Lightweight Underwater Object Detection Based on YOLO v4 and Multi-Scale Attentional Feature Fusion," *Remote Sens.*, vol. 13, no. 22, p. 4706, Nov. 2021, doi: 10.3390/rs13224706.

[65] Z. Zakria, J. Deng, R. Kumar, M. S. Khokhar, J. Cai, and J. Kumar, "Multiscale and Direction Target Detecting in Remote Sensing Images via Modified YOLO-v4," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, vol. 15, pp. 1039–1048, 2022, doi: 10.1109/JSTARS.2022.3140776.

[66] T. Liu, B. Pang, L. Zhang, W. Yang, and X. Sun, "Sea Surface Object Detection Algorithm Based on YOLO v4 Fused with Reverse Depthwise Separable Convolution (RDSC) for USV," *J. Mar. Sci. Eng.*, vol. 9, no. 7, p. 753, Jul. 2021, doi: 10.3390/jmse9070753.

[67] A. I. B. Parico and T. Ahamed, "Real Time Pear Fruit Detection and Counting Using YOLOv4 Models and Deep SORT," *Sensors*, vol. 21, no. 14, p. 4803, Jul. 2021, doi: 10.3390/s21144803.

[68] P. Shi *et al.*, "Oil Well Detection via Large-Scale and High-Resolution Remote Sensing Images Based on Improved YOLO v4," *Remote Sens.*, vol. 13, no. 16, p. 3243, Aug. 2021, doi: 10.3390/rs13163243.

[69] C. Zhang, T. Li, and W. Zhang, "The Detection of Impurity Content in Machine-Picked Seed Cotton Based on Image Processing and Improved YOLO V4," *Agronomy*, vol. 12, no. 1, p. 66, Dec. 2021, doi: 10.3390/agronomy12010066.

[70] L. Ting, Z. Baijun, Z. Yongsheng, and Y. Shun, "Ship Detection Algorithm based on Improved YOLO V5," in *2021 6th International Conference on Automation, Control and Robotics Engineering (CACRE)*, IEEE, Jul. 2021, pp. 483–487. doi: 10.1109/CACRE52464.2021.9501331.

[71] Z. Chen *et al.*, "Plant Disease Recognition Model Based on Improved YOLOv5," *Agronomy*, vol. 12, no. 2, p. 365, Jan. 2022, doi: 10.3390/agronomy12020365.

[72] Z. Zhao, X. Yang, Y. Zhou, Q. Sun, Z. Ge, and D. Liu, "Real-time detection of particleboard surface defects based on improved YOLOV5 target detection," *Sci. Rep.*, vol. 11, no. 1, p. 21777, Nov. 2021, doi: 10.1038/s41598-021-01084-x.

[73] H. Zhang, M. Tian, G. Shao, J. Cheng, and J. Liu, "Target Detection of Forward-Looking Sonar Image Based on Improved YOLOv5," *IEEE Access*, vol. 10, pp. 18023–18034, 2022, doi: 10.1109/ACCESS.2022.3150339.

[74] B. Yan, P. Fan, X. Lei, Z. Liu, and F. Yang, "A Real-Time Apple Targets

Detection Method for Picking Robot Based on Improved YOLOv5," *Remote Sens.*, vol. 13, no. 9, p. 1619, Apr. 2021, doi: 10.3390/rs13091619.

[75] Q. Wang, M. Cheng, S. Huang, Z. Cai, J. Zhang, and H. Yuan, "A deep learning approach incorporating YOLO v5 and attention mechanisms for field real-time detection of the invasive weed Solanum rostratum Dunal seedlings," *Comput. Electron. Agric.*, vol. 199, p. 107194, Aug. 2022, doi: 10.1016/j.compag.2022.107194.

[76] J. Qi *et al.*, "An improved YOLOv5 model based on visual attention mechanism: Application to recognition of tomato virus disease," *Comput. Electron. Agric.*, vol. 194, p. 106780, Mar. 2022, doi: 10.1016/j.compag.2022.106780.

[77] Y. Yu, J. Zhao, Q. Gong, C. Huang, G. Zheng, and J. Ma, "Real-Time Underwater Maritime Object Detection in Side-Scan Sonar Images Based on Transformer-YOLOv5," *Remote Sens.*, vol. 13, no. 18, p. 3555, Sep. 2021, doi: 10.3390/rs13183555.

[78] Y. Zhang, Z. Guo, J. Wu, Y. Tian, H. Tang, and X. Guo, "Real-Time Vehicle Detection Based on Improved YOLO v5," *Sustainability*, vol. 14, no. 19, p. 12274, Sep. 2022, doi: 10.3390/su141912274.

[79] T.-H. Wu, T.-W. Wang, and Y.-Q. Liu, "Real-Time Vehicle and Distance Detection Based on Improved Yolo v5 Network," in *2021 3rd World Symposium on Artificial Intelligence (WSAI)*, IEEE, Jun. 2021, pp. 24–28. doi: 10.1109/WSAI51899.2021.9486316.

[80] J. Zhao *et al.*, "A Wheat Spike Detection Method in UAV Images Based on Improved YOLOv5," *Remote Sens.*, vol. 13, no. 16, p. 3095, Aug. 2021, doi: 10.3390/rs13163095.

[81] S. Luo, J. Yu, Y. Xi, and X. Liao, "Aircraft Target Detection in Remote Sensing Images Based on Improved YOLOv5," *IEEE Access*, vol. 10, pp. 5184–5192, 2022, doi: 10.1109/ACCESS.2022.3140876.

[82] H. Wang, S. Zhang, S. Zhao, Q. Wang, D. Li, and R. Zhao, "Real-time detection and tracking of fish abnormal behavior based on improved YOLOV5 and SiamRPN++," *Comput. Electron. Agric.*, vol. 192, p. 106512, Jan. 2022, doi: 10.1016/j.compag.2021.106512.

[83] C. Li *et al.*, "YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications," Sep. 2022.

[84] C. Li *et al.*, "YOLOv6 v3.0: A Full-Scale Reloading," Jan. 2023, [Online]. Available: http://arxiv.org/abs/2301.05586

[85] K. Weng, X. Chu, X. Xu, J. Huang, and X. Wei, "EfficientRep:An Efficient Repvgg-style ConvNets with Hardware-aware Neural Network Design," Feb. 2023, [Online]. Available: http://arxiv.org/abs/2302.00386

[86] Y. Xia, M. Nguyen, and W. Q. Yan, "A Real-Time Kiwifruit Detection Based on Improved YOLOv7," 2023, pp. 48–61. doi: 10.1007/978-3-031-25825-1_4.

[87] K. Jiang *et al.*, "An Attention Mechanism-Improved YOLOv7 Object Detection Algorithm for Hemp Duck Count Estimation," *Agriculture*, vol. 12, no. 10, p. 1659, Oct. 2022, doi: 10.3390/agriculture12101659.

[88] K. Liu, Q. Sun, D. Sun, M. Yang, and N. Wang, "Underwater target detection based on improved YOLOv7," Feb. 2023, [Online]. Available: http://arxiv.org/abs/2302.06939

[89] J. Zheng, H. Wu, H. Zhang, Z. Wang, and W. Xu, "Insulator-Defect Detection Algorithm Based on Improved YOLOv7," *Sensors*, vol. 22, no. 22, p. 8801, Nov. 2022, doi: 10.3390/s22228801.

[90] Y. Zhang, Y. Sun, Z. Wang, and Y. Jiang, "YOLOv7-RAR for Urban Vehicle Detection," *Sensors*, vol. 23, no. 4, p. 1801, Feb. 2023, doi: 10.3390/s23041801.

[91] Y. Lai, R. Ma, Y. Chen, T. Wan, R. Jiao, and H. He, "A Pineapple Target Detection Method in a Field Environment Based on Improved YOLOv7," *Appl. Sci.*, vol. 13, no. 4, p. 2691, Feb. 2023, doi: 10.3390/app13042691.

[92] Y. Qiu, Y. Lu, Y. Wang, and H. Jiang, "IDOD-YOLOV7: Image-Dehazing YOLOV7 for Object Detection in Low-Light Foggy Traffic Environments," *Sensors*, vol. 23, no. 3, p. 1347, Jan. 2023, doi: 10.3390/s23031347.

[93] C. Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall, "Activation Functions: Comparison of trends in Practice and Research for Deep Learning," Nov. 2018, doi: 1811.03378.

[94] D. Misra, "Mish: A Self Regularized Non-Monotonic Activation Function," Aug. 2019, [Online]. Available: http://arxiv.org/abs/1908.08681

[95] Q. Wang, Y. Ma, K. Zhao, and Y. Tian, "A Comprehensive Survey of Loss Functions in Machine Learning," *Ann. Data Sci.*, vol. 9, no. 2, pp. 187–212, Apr. 2022, doi: 10.1007/s40745-020-00253-5.

[96] L. N. Smith, "A disciplined approach to neural network hyper-parameters: Part 1 -- learning rate, batch size, momentum, and weight decay," Mar. 2018, doi: 1803.09820.

[97] S. Postalcıoğlu, "Performance Analysis of Different Optimizers for Deep Learning-Based Image Recognition," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 34, no. 02, p. 2051003, Feb. 2020, doi: 10.1142/S0218001420510039.

[98] "Drone Dataset (UAV) | Kaggle." https://www.kaggle.com/datasets/dasmehdixtr/drone-dataset-uav (accessed Jun. 12, 2023).